# Stacks

# STACKS

- **Stack** is an abstract data type with a bounded(predefined) capacity. It is a simple data structure that allows **adding and removing elements** in a particular order.
- A stack is an **abstract data type** that holds an ordered, linear sequence of items. A stack is a **last in, first out** (LIFO) structure.
- Every time an element is added, it goes on the **top** of the stack and the only element that can be removed is the element that is at the top of the stack.

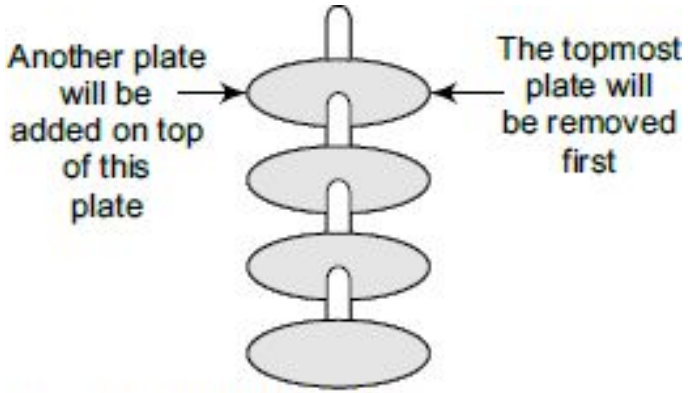
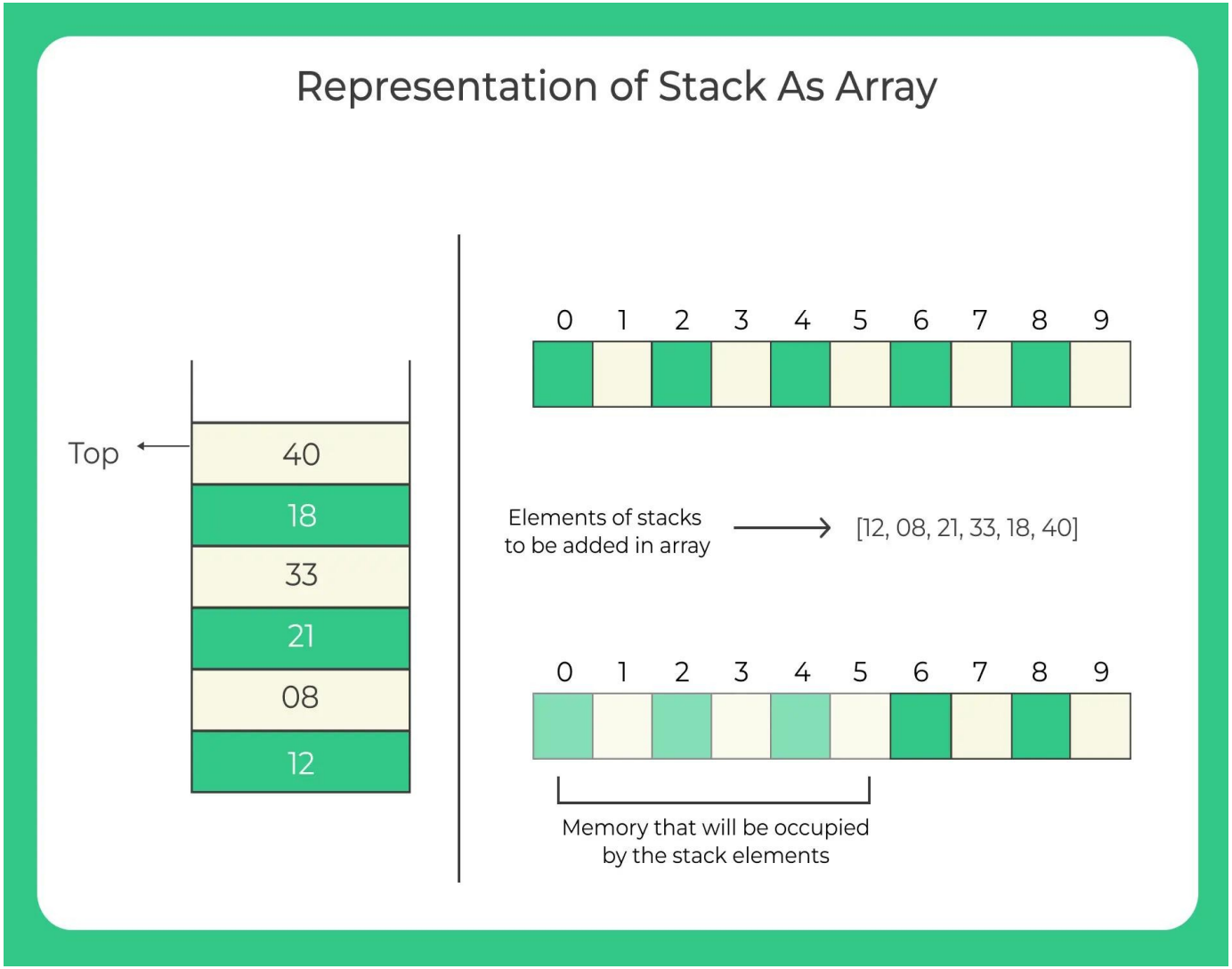



**Figure 7.1** Stack of plates

# Operations on stack

| push(data) | adds an element to the top of the stack |
|---|---|
| pop() | removes an element from the top of the stack |
| peek() | returns a copy of the element on the top of the stack without removing it |
| is_empty() | checks whether a stack is empty |
| is_full() | checks whether a stack is at maximum capacity when stored in a static (fixed-size) structure |

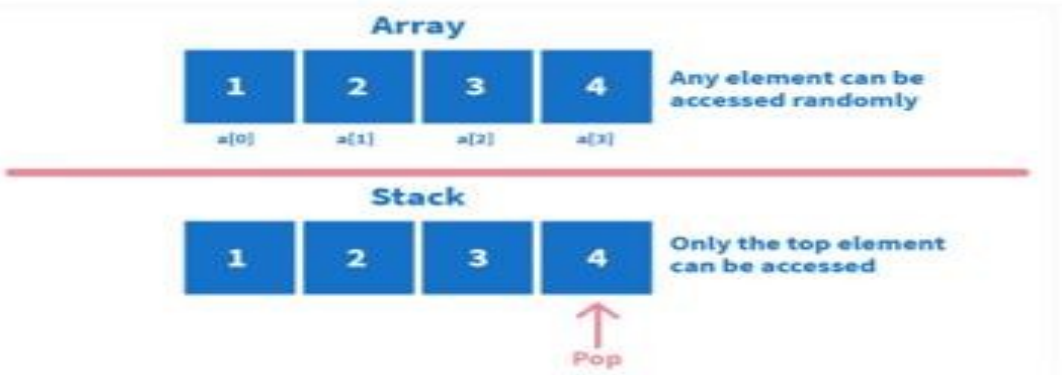
# Implementation of Stack



# Stack VS Array

Stacks differ from the arrays in a way that in arrays random access is possible; this means that we can access any element just by using its index in the array. Whereas, in a stack **only limited access is possible and only the top element is directly available.**

Consider an array, arr = [1, 2, 3, 4]. To access 2, we can simply write arr[1], where 1 is the index of the array element, 2. But if the same is implemented using a stack, we can only access the topmost element that is 4.



Vivekananda Institute of Professional Studies

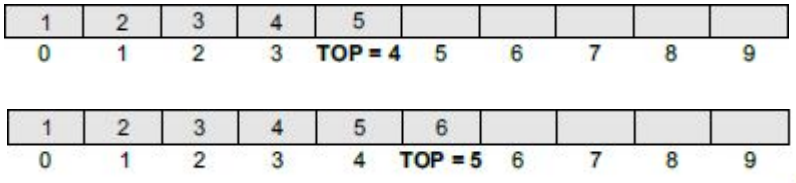
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

# Operation on Stack using Arrays

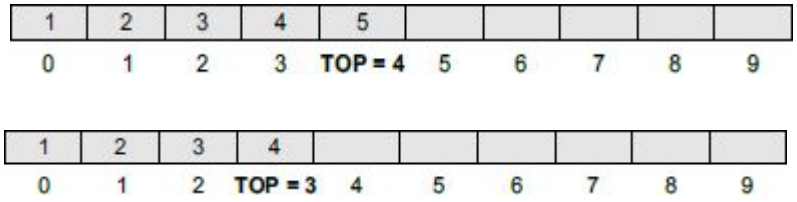
## Push

```
void push(int st[], int val)
{
        if(top == MAX-1)
        {
                printf("\n STACK OVERFLOW");
        }
        else
        {
                top++;
                st[top] = val;
        }
}
```

```
Step 1: IF TOP = MAX-1
            PRINT "OVERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET TOP = TOP + 1
Step 3: SET STACK[TOP] = VALUE
Step 4: END
```



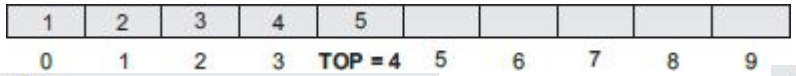

## Pop




```
int pop(int st[])
{
        int val;
        if(top == -1)
        {
                printf("\n STACK UNDERFLOW");
                return -1;
        }
        else
        {
                val = st[top];
                top--;
                return val;
        }
}
```

```
Step 1: IF TOP = NULL
            PRINT "UNDERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: END
```

## Peek




```
int peek(int st[])
{
        if(top == -1)
        {
                printf("\n STACK IS EMPTY");
                return -1;
        }
        else
        return (st[top]);
}
```

```
Step 1: IF TOP = NULL
            PRINT "STACK IS EMPTY"
            Goto Step 3
Step 2: RETURN STACK[TOP]
Step 3: END
```

## Travers e

```
void display(int st[])
{
        int i;
        if(top == -1)
        printf("\n STACK IS EMPTY");
        else
        {
                for(i=top;i>=0;i--)
                printf("\n %d",st[i]);
                printf("\n"); // Added for formatting purposes
        }
}
```

**Push O(1), Pop O(1),Peek O(1) and Traverse O(n)**

# Stack implementation in C

```
#include <stdio.h>
int stack[100],i,j,choice=0,n,top=-1;
void push();
void pop();
void show();
void main ()
{

   printf("Enter the number of elements in the stack ");
   scanf("%d",&n);
   printf("*********Stack operations using array*********");

printf("\n----------------------------------------\n");
   while(choice != 4)
   {
      printf("Chose one from the below options...\n");
      printf("\n1.Push\n2.Pop\n3.Show\n4.Exit");
      printf("\n Enter your choice \n");
      scanf("%d",&choice);
```

# Stack implementation in C

```
switch(choice)
      {
         case 1:
            push();
            break;
         case 2:
            pop();
            break;
         case 3:
            show();
            break;
         case 4:
            printf("Exiting....");
            break;
         default:
         {
            printf("Please Enter valid choice ");
         }
      }
   }
}
```

# Stack implementation in C

```
switch(choice)
      {
         case 1:
            push();
            break;
         case 2:
            pop();
            break;
         case 3:
            show();
            break;
         case 4:
            printf("Exiting....");
            break;
         default:
         {
            printf("Please Enter valid choice ");
         }
      };
   }
}
```

# Stack implementation in C

```
void push ()
{
   int val;
   if (top == n-1 )
   printf("\n Overflow");
   else
   {
      printf("Enter the value?");
      scanf("%d",&val);
      top = top +1;
      stack[top] = val;
   }
}

void pop ()
{
   if(top == -1)
   printf("Underflow");
   else
   top = top -1;
}
```

# Stack implementation in C

```
void show()
{
   for (i=top;i>=0;i--)
   {
      printf("%d\n",stack[i]);
   }
   if(top == -1)
   {
      printf("Stack is empty");
   }
}
```

# Need of Stack

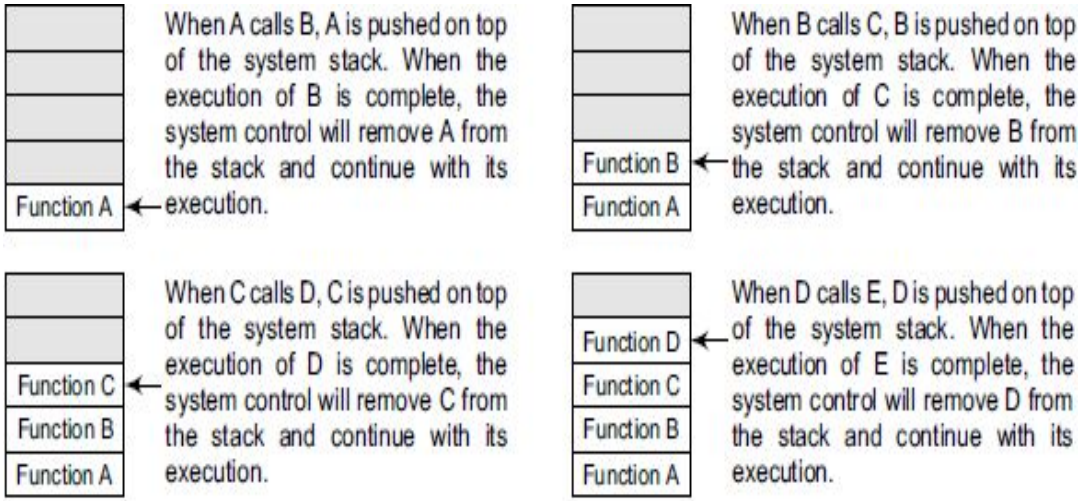



**Figure 7.2** System stack in the case of function calls

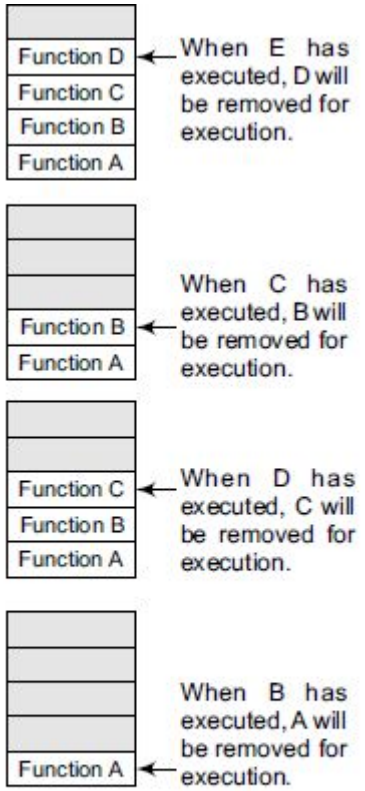



**Figure 7.3** System stack when a called function returns to the calling function

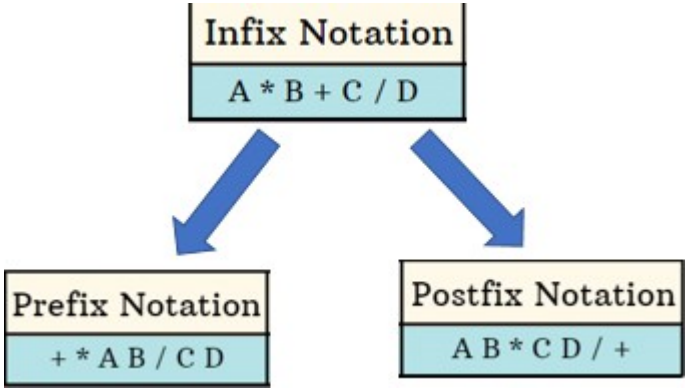
# Applications

- **Function calls and recursion:** When a function is called, the current state of the program is pushed onto the stack. When the function returns, the state is popped from the stack to resume the previous function's execution.
- **Undo/Redo operations:** The undo-redo feature in various applications uses stacks to keep track of the previous actions. Each time an action is performed, it is pushed onto the stack. To undo the action, the top element of the stack is popped, and the reverse operation is performed.
- **Expression evaluation:** Stack data structure is used to evaluate expressions in infix, postfix, and prefix notations. Operators and operands are pushed onto the stack, and operations are performed based on the stack's top elements.
- **Browser history:** Web browsers use stacks to keep track of the web pages you visit. Each time you visit a new page, the URL is pushed onto the stack, and when you hit the back button, the previous URL is popped from the stack.
- **Balanced Parentheses:** Stack data structure is used to check if parentheses are balanced or not. An opening parenthesis is pushed onto the stack, and a closing parenthesis is popped from the stack. If the stack is empty at the end of the expression, the parentheses are balanced.
- **Backtracking Algorithms:** The backtracking algorithm uses stacks to keep track of the states of the problem-solving process. The current state is pushed onto the stack, and when the algorithm backtracks, the previous state is popped from the stack.

# Arithmetic Expression

**Infix notation** : <operands><operator><operands>
**Postfix notation (Reverse Polish notation)**: <operands> <operands> <operator>

**Prefix notation (Polish notation)**:<operator><operands><operands>


Binary operations have different levels of precedence and association .

- First : Exponentiation (^)
- Second: Multiplication (*) and Division (/)
- Third : Addition (+) and Subtraction (-)

# Evaluation of Arithmetic Expression

- Evaluate the following Arithmetic Expression:

  5 ^ 2 + 3 * 5 – 6 * 2 / 3 + 24 / 3 + 3

- First:

  25 + 3 * 5 – 6 * 2 / 3 + 24 / 3 + 3

- Second:

  25 + 15 – 4 + 8 + 3

- Third:

  47

# Evaluation of Postfix Expression

❑P is an arithmetic expression in Postfix Notation.

1. Add a right parenthesis ")" at the end of P.

2. Scan P from left to right and Repeat Step 3 and 4 for each element of P until the sentinel ")" is encountered.

3. If an operand is encountered, put it on STACK.

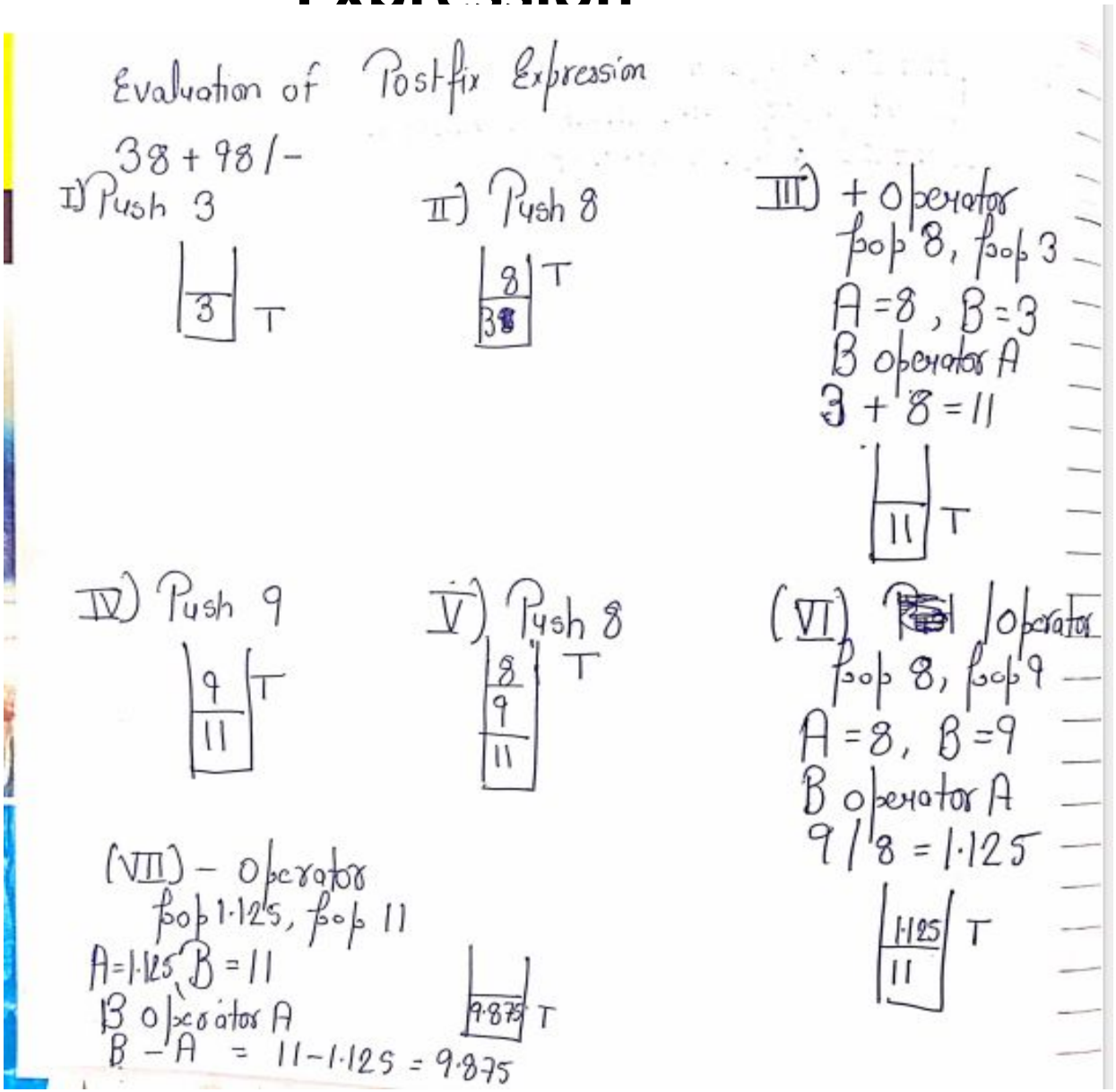
4. If an operator @ is encountered, then:
   (a) Remove the two top elements of STACK, where A is the top element and B is the next to top element.
   (b) Evaluate B @ A.
   (c) Place the result of (B) back on STACK.
   **[End of if structure.]**
   **[End of step 2 Loop.]**

5. Set VALUE equal to the top element on STACK.

6. Exit.

VIPS
Vivekananda Institute of Professional Studies
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

# Evaluation of Postfix Expression



# Evaluation of Postfix Expression

Contemplate the postfix expression 234*+ as an example.

| Input | Stack | Postfix evaluation |
|---|---|---|
| 2 3 4 * + | *empty* | Push 2 into stack |
| 3 4 * + | 2 | Push 3 into stack |
| 4 * + | 3 2 | Push 4 into stack |
| * + | 4 3 2 | Pop 4 & pop 3 from stack and do 3 *4 , push 12 into stack |
| + | 12 2 | Pop 12 & Pop 2 from stack do 2 + 12, push 14 into stack |
| | 14 | |

VIPS
Vivekananda Institute of Professional Studies
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

# Evaluation of Postfix Expression

3 8 + 9 8 / -

| | |
|---|---|
| 3 8 | 8<br>3 |
| + | 3 + 8 = 11<br>11 |
| 9 8 | 8<br>9<br>11 |
| / | 9/8 = 1.125<br>1.125<br>11 |
| - | 11 – 1.125 = 9.875<br>9.875 |

VIPS Vivekananda Institute of Professional Studies
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

# Evaluation of Postfix Expression

## PRACTICE QUESTION

2 3 4 + 5 6 - - *

23-4+567*+*

# Evaluation of Prefix Expression

**Step 1:** Start from the last element of the expression.
**Step 2:** check the current element.
**Step2.1:** if it is an operand, push it to the stack. **Step 2.2:** If it is an operator, pop two operands from the stack. A is top element and B is next to top element. Perform the operation (A op B) and push the result back to the stack.
**Step 3:** Do this till all the elements of the expression are traversed and return the top of stack which will be the result of the operation.

VIPS Vivekananda Institute of Professional Studies
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

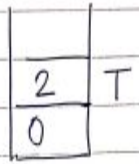
# Evaluation of Prefix Expression

Evaluation of Prefix Expression

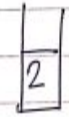
-+7*45+20

I) Push 0

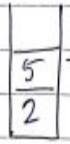

II) Push 2



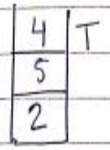

III) +operator
pop 0, pop 2
A=2, B=0
A operator B
A+B
2+0=2



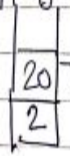

IV) Push 5



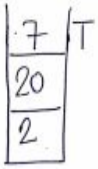

V Push 4



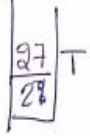

VI) *operator
pop 4, pop 5
A=4, B=5
A*B=20



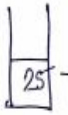

VII. Push 7




VIII) +operator
pop 7, pop 20
A=7, B=20
A+B = 7+20 = 27




IX) - operator
pop 27, pop 2
A=27, B=2




A operator B = A-B
27-2
25

# Evaluation of Prefix Expression

```
Expression: * 4 5
Output: 20
```

**Explanation Table:**

| Scanned Character | Stack | Explanation |
|---|---|---|
| 4 | 4 | 4 is an operand, push it into the stack |
| 5 | 4 5 | 5 is an operand, push it into the stack |
| * ( asterisks) | 20 (5 * 4) | * is an operator, pop 5 and 4, multiply them and push the result into stack |
| **Result** | **20** | **20 is the final answer** |

# Evaluation of Prefix Expression

## PRACTICE QUESTIONS

```
Input : -+7*45+20
```

```
* + 6 9 - 3 1
```

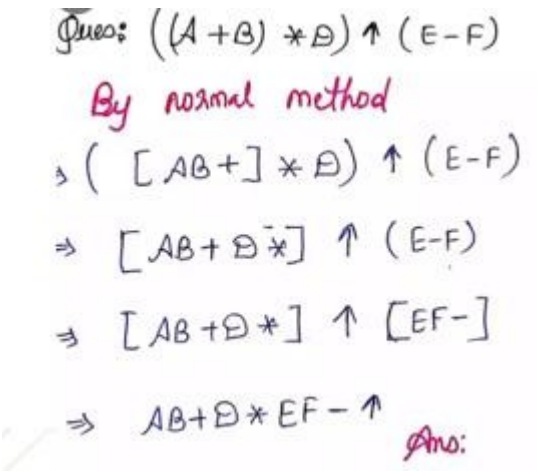
```
Input : -+8/632
Output : 8

Input : -+7*45+20
Output : 25
```

Vivekananda Institute of Professional Studies

योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

# Infix to Postfix Conversion

Method 1:




- A + B * C
- → (A + (B * C))
- → (A + (B C *) )
- → A B C * +

- A + B * C + D
- → ((A + (B * C)) + D )
- → ((A + (B C*) )+ D)
- → ((A B C *+) + D)
- → A B C * + D +

# Infix to Postfix Conversion using Stack

```
Step 1: Add ")" to the end of the infix expression
Step 2: Push "(" on to the stack
Step 3: Repeat until each character in the infix notation is scanned
        IF a "(" is encountered, push it on the stack
        IF an operand (whether a digit or a character) is encountered, add it to the
        postfix expression.
        IF a ")" is encountered, then
          a. Repeatedly pop from stack and add it to the postfix expression until a
             "(" is encountered.
          b. Discard the "(". That is, remove the "(" from stack and do not
             add it to the postfix expression
        IF an operator 0 is encountered, then
          a. Repeatedly pop from stack and add each operator (popped from the stack) to the
             postfix expression which has the same precedence or a higher precedence than 0
          b. Push the operator 0 to the stack
        [END OF IF]
Step 4: Repeatedly pop from the stack and add it to the postfix expression until the stack is empty
Step 5: EXIT
```

# Infix to Postfix

Infix to Postfix

(A+B) * C/D + E ^ A / B

Insert '(' and ')' ((A+B) * C/D + E ^ A / B)

| Input Symbol | Stack | Postfix Expression |
|---|---|---|
| ( | ( | |
| ( | (( | |
| A | (( | A |
| + | ((+ | A |
| B | ((+ | AB |
| ) | (+) pop | AB+ |
| * | (* | AB+ |
| C | (* | AB+C |
| / | (*/ pop | AB+C* |
| D | (/ | AB+C*D |
| + | (+ pop | AB+C*D/ |
| E | (+ | AB+C*D/E |
| ^ | (+^ | AB+C*D/E |
| A | (+^ | AB+C*D/EA |
| / | (+^/ pop | AB+C*D/EA^ |
| B | (+/ | AB+C*D/EA^B |
| ) | (+( pop | AB+C*D/EA^B/+ |

# Infix to Postfix

(a) A - (B / C + (D % E * F) / G)* H

(b) A - (B / C + (D % E * F) / G)* H)

# Infix to Prefix

1. Push ) on to stack and add ( to the end of expression
2. Scan infix expression from Right to left and repeat steps 3 to 6 for each character of expression until stack is empty.
3. If the current character is an operand ,append it to the output string.
4. If the current character is a closing bracket ‘)’, push it onto the stack.
5. If the current character is an operator then

A) Repeatedly pop from stack and add to the expression , each operator which has higher precedence than the current operator.

B) Push current operator to the stack.

6) If the current character is an opening bracket ‘(‘, pop the operators from the stack and append them to the output string until the corresponding closing bracket ‘)’ is encountered. Discard the closing bracket.

7) Reverse the output string to obtain the prefix expression.

## Infix to Prefix Conversion

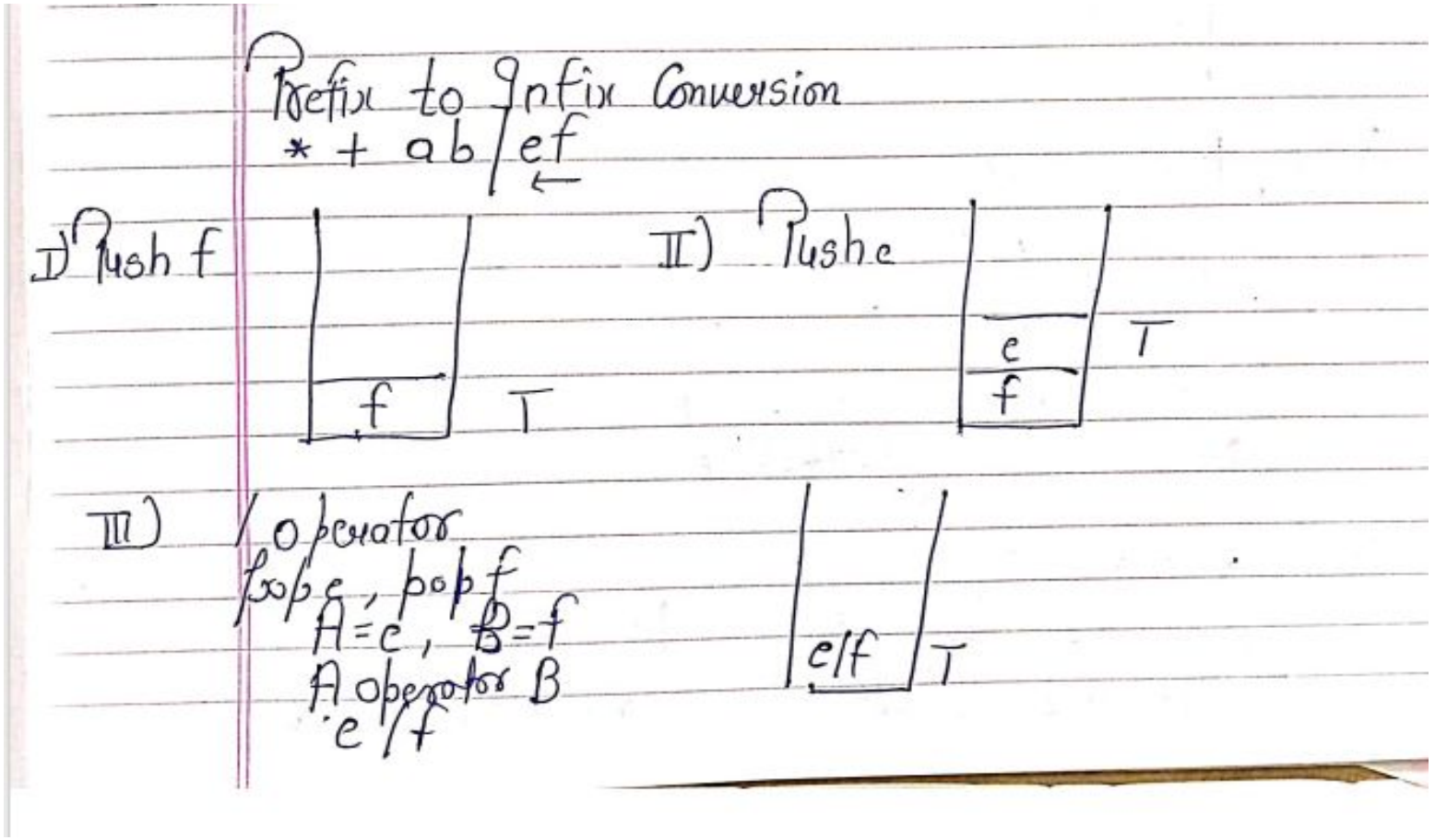
Infix - (P + (Q * R) / (S - T))

Reverse: ) ) T - S ( / ) R * Q ( + P (

| Input Symbol | Stack | Output / Prefix Expression |
|---|---|---|
| ) | ) | - |
| ) | ) ) | - |
| T | ) ) | T |
| - | ) ) - | T |
| S | ) ) - | T S |
| ( | ) ) - ( pop | T S - |
| / | ) / | T S - |
| ) | ) / ) | T S - |
| R | ) / ) | T S - R |
| * | ) / ) * | T S - R |
| Q | ) / ) * | T S - R Q |
| ( | ) / ) * ( pop | T S - R Q * |
| + | ) / + pop | T S - R Q * / |
| P | ) + | T S - R Q * / P |
| ( | ) + ( pop | T S - R Q * / P + |

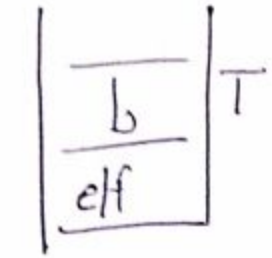
Prefix Expression: +P/*QR-ST

# Prefix to infix

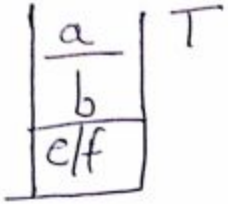


# Prefix to infix

IV) Push b




V) Push a



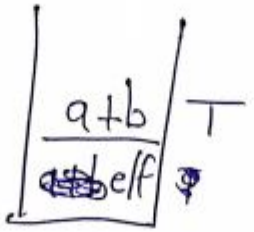

VI) + operator

pop a, pop b

A = a, B = b

A operator B

a + b



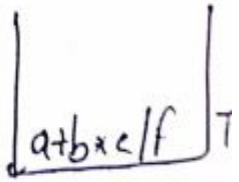

VII) * operator

pop a+b, pop elf

A = a+b, B = elf

A operator B

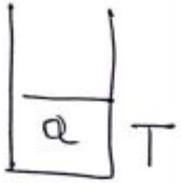
a+b * elf



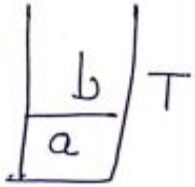
# Postfix to infix

Postfix to Infix
ab+ef/*

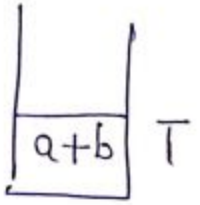
(I) Push a



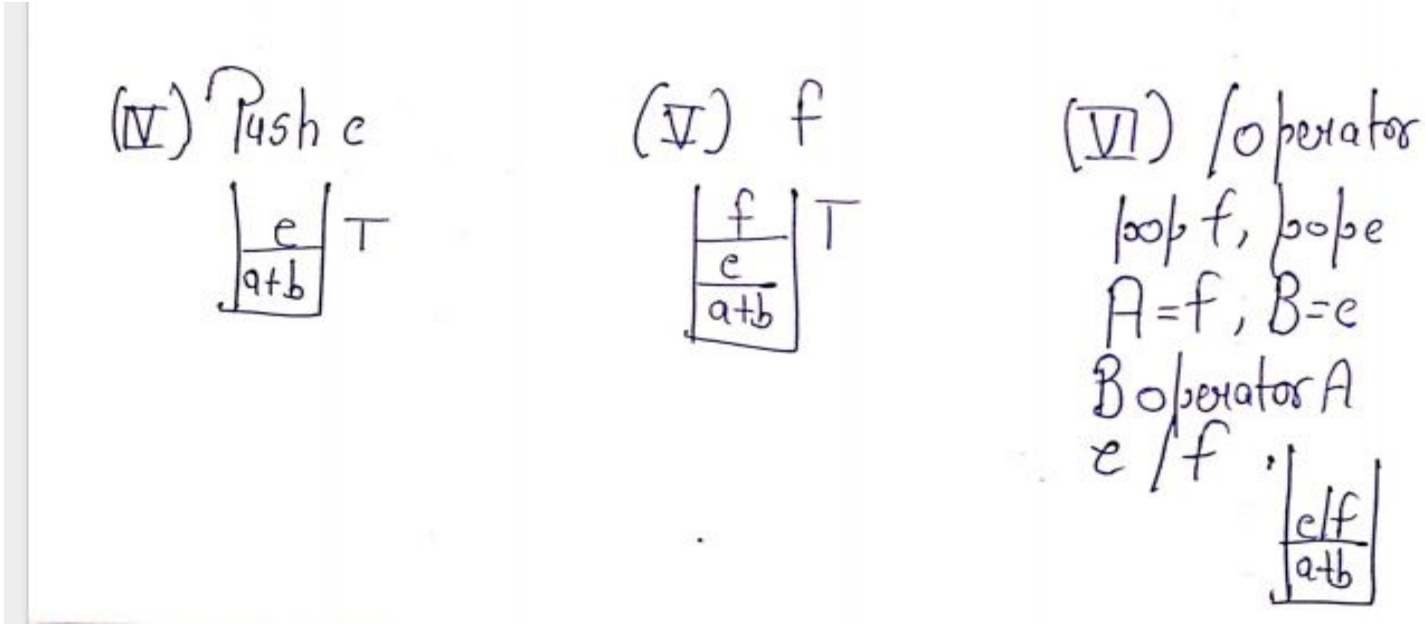

(II) Push b



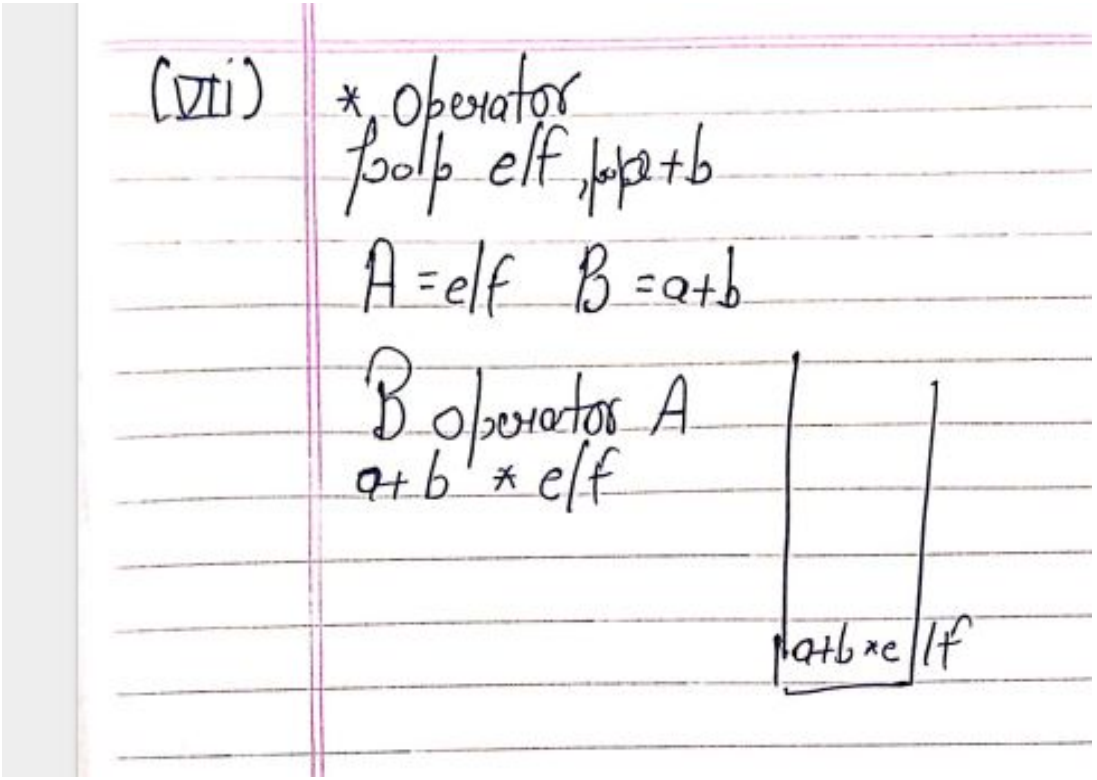

(III) + operator
pop b, pop a
A = b, B = a
B operator A
a + b



# Postfix to infix



# Postfix to infix



# Recursion

A recursive function is defined as a function that calls itself to solve a smaller version of its task until a final call is made which does not require a call to itself. Since a recursive function repeatedly calls itself, it makes use of the system stack to temporarily store the return address and local variables of the calling function. Every recursive solution has two major cases. They are

1. Base case, in which the problem is simple enough to be solved directly without making any further calls to the same function.

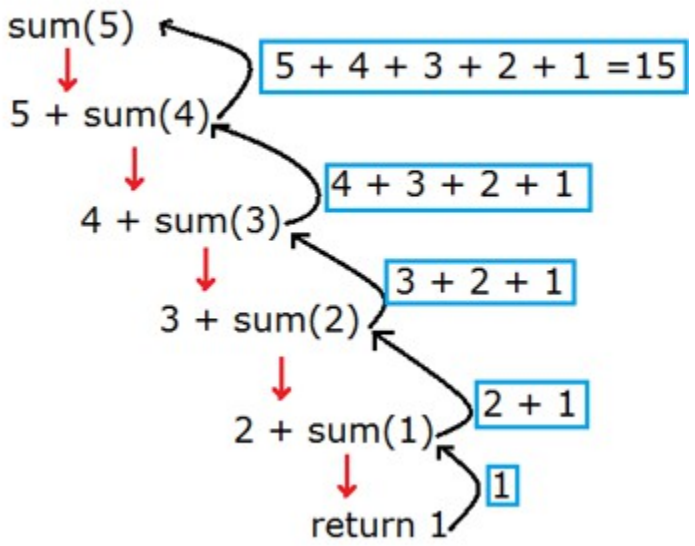
2. Recursive case, in which first the problem at hand is divided into simpler sub-parts. Second the function calls itself but with sub-parts of the problem obtained in the first step. Third, the result is obtained by combining the solutions of simpler sub-parts.

Every recursive function must have at least one base case. Otherwise, the recursive function will generate an infinite sequence of calls, thereby resulting in an error condition known as an infinite stack.

# Sum of natural number using recursion

This exit condition inside a recursive function is known as base condition.

```
int sum(int n)
{
  if(n==1)  //Base Condition
    return 1;
  return n+ sum(n-1); // Function calling itself
}
```



# Types of Recursion

- Direct vs Indirect recursion :

A function is said to be *directly* recursive if it explicitly calls itself.

A function is said to be *indirectly* recursive if it contains a call to another function which ultimately calls it.

```
int Func (int n)
{
    if (n == 0)
        return n;
    else
        return (Func (n-1));
}
```

Figure 7.28 Direct recursion

```
int Func1 (int n)
{
    if (n == 0)
        return n;
    else
        return Func2(n);
}
int Func2(int x)
{
     return Func1(x-1);
}
```

Figure 7.29 Indirect recursion

- LinearVs Tree recursive function : If a function calling itself for one time then it's known as Linear recursive Recursion. Otherwise if a recursive function calling itself for more than one time then it's known as Tree Recursion. For example :

Linear : factorial Tree: fibnoacci

VIPS
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

Vivekananda Institute
of Professional Studies

VIPS
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

Vivekananda Institute
of Professional Studies

END